

ESE WEB TECH PAPER MORNING SHIFT

Instruction:

1. No internet will be allowed if any student does any unfair activity, then strictly there will be UFM.
2. There are 4 question each of 20 marks you have to do 3 questions from it.

Q.1 Complete the Given ES6 Driver Code. (20 Marks)

//Copy Code From Here

```
const students = [  
  {  
    name: "Amit",  
    marks: [78, 90, 66],  
    skills: ["JS", "React", "HTML", "CSS"],  
    address: { city: "Ghaziabad", state: "UP" }  
  },  
  {  
    name: "Riya",  
    marks: [88, 95, 70],  
    skills: ["Node", "MongoDB", "Flutter"],  
    address: null  
  },  
  {  
    name: "Karan",  
    marks: [45, 55, 60],
```

```
    skills: ["Python", "Flask"],  
    address: { city: "Delhi", state: "Delhi" }  
  }  
];  
const courses = ["JS", "React", "Node", "Express"];
```

//Q1 Return first student Name

// Output -> Amit

```
function firstStudentName(arr){  
  
}
```

// Q2 Return count of students having average greater than 70

// Output -> 2

```
function studentCount(arr){  
  
}
```

// Q3 Return all courses with their string lengths

//Output -> [['JS',2],['React',5],['Node',4],['Express',7]]

```
function allCourses(arr){  
  
}
```

// Q4 Return first and last course using destructuring

//Output -> { firstCourse: 'JS', lastCourse: 'Express' }

```
function firstCourse(arr){  
  
}
```

// Q5 Return greeting with average marks using template literal

// Of First Student

// Output: Hello Amit, your average marks are 78

```
function greetStudent(arr){  
}
```

// Q6 Return uppercase student names with total marks

//[['AMIT',234],['RIYA',253],['KARAN',160]]

```
function upperNames(arr){  
}
```

// Q7 Return students whose every mark is above 65

//["Amit","Riya"]

```
function toppers(arr){  
}
```

// Q8 : Return all student cities if city not available

// then Not Available

// Output -> ['Ghaziabad','Not Available','Delhi']

```
function studentCity(arr){  
}
```

// Q9 Return city-state string or "City Not Found"

// ['Ghaziabad-UP', 'City Not Found', 'Delhi-Delhi']

```
function safeCity(arr){  
}
```

// Q10 Destructure first course and count remaining courses

// Output : { firstCourse: 'JS', remainingCourses: 3 }

```
function courseInfo(arr){  
  
}
```

// Q11 Merge marks of first and second student and sort ascending

// Output : [66, 70, 78, 88, 90, 95]

```
function mergeMarks(arr){
```

```
}
```

// Q12 Return average marks of first student using reduce()

// Output 78

```
function totalMarks(arr){
```

```
}
```

// Q13 Return student names sorted by average marks descending

// Output : ['Riya', 'Amit', 'Karan']

```
function averageMarksDesc(arr){
```

```
}
```

// Q14 Return Promise: after 2 sec if all students have

// average marks above 60 then Resolve All Students Performing

// Well otherwise Reject Some Students Need Improvement

//Output : Some Students Need Improvement

```
function performancePromise(arr){
```

```
}
```

// Q15 Consume performancePromise() using then() and catch()

// return result in lowercase

// Output: some students need improvement

```
function performanceReport(arr){
```

```
}
```

```
// DRIVER FUNCTION DO NOT MODIFY
function driver(){

  console.log("Q1:", firstStudentName(students));
  console.log("Q2:", studentCount(students));
  console.log("Q3:", allCourses(courses));
  console.log("Q4:", firstCourse(courses));
  console.log("Q5:", greetStudent(students));

  console.log("Q6:", upperNames(students));
  console.log("Q7:", toppers(students));
  console.log("Q8:", studentCity(students));
  console.log("Q9:", safeCity(students));
  console.log("Q10:", courseInfo(courses));

  console.log("Q11:", mergeMarks(students));
  console.log("Q12:", totalMarks(students));
  console.log("Q13:", averageMarksDesc(students));

  performancePromise(students)
    .then(res => console.log("Q14:", res))
    .catch(err => console.log("Q14:", err));

  performanceReport(students)
    .then(res => console.log("Q15:", res))
    .catch(err => console.log("Q15:", err));
}

driver();
//Copy Till Here
```

Q.2 Complete the Given Dart Driver Code.

(20 Marks)

//Copy From Here

```
List<int?> numbers = [10, null, -5, 25, 18, null, 7, 40];
```

```
List<String> subjects = ["Flutter", "Dart", "Firebase"];
```

```
class Student {  
  String name;  
  int rollNo;  
  double cgpa;  
  String branch;  
  String city;  
  Student(this.name, this.rollNo, this.cgpa, this.branch, this.city);  
}
```

```
class Topper extends Student {  
  double scholarship;  
  Topper(  
    String name,  
    int rollNo,  
    double cgpa,  
    String branch,  
    String city,  
    this.scholarship,  
  ) : super(name, rollNo, cgpa, branch, city);  
}
```

```
mixin Message {  
  String showMessage() {  
    return "Welcome Student";  
  }  
}
```

```
class College with Message {}
```

```
Student stu = Student("Ramesh", 101, 8.7, "CSE", "Jaipur");
```

```
// Q1 Return total number of subjects
```

```
// Output -> 3
```

```
int subjectCount(List<String> arr) {}
```

```
// Q2 Return first subject in uppercase
```

```
// Output -> FLUTTER
```

```
String firstSubject(List<String> arr) {}
```

```
// Q3 Return student city using object
```

```
// Output -> Jaipur
```

```
String studentCity(Student s) {}
```

```
// Q4 Return square of all non-null positive numbers
```

```
// Output -> [100, 625, 324, 49, 1600]
```

```
List<int> squareNumbers(List<int?> arr) {}
```

```
// Q5 Medium: Return total of numbers
```

```
// if null add 0
```

```
// if negative convert into positive then add
```

```
// if positive add as it is
```

```
// Output -> 105
```

```
int totalNumbers(List<int?> arr) {}
```

```
// Q6 Medium: Return all even numbers greater than 15
```

```
// Output -> [18, 40]
```

```
List<int> evenGreaterThan15(List<int?> arr) {}
```

```
// Q7 Calculator using switch case
// operations -> add, sub, mul, div
// Output -> 200
double calculator(double a, double b, String op) {}
```

```
// Q8 Return sentence using string interpolation
// Output -> Student Ramesh belongs to CSE
String studentInfo(Student s) {}
```

```
// Q9 Return CGPA status
// if cgpa > 8 return Excellent else Average
// Output -> Excellent
String cgpaStatus(Student s) {}
```

```
// Q10 Create Topper object and return scholarship amount
// Output -> 50000.0
double scholarshipAmount() {}
```

```
// Q.11 Use mixin through College class
// Output -> Welcome Student
String mixinDemo() {}
```

```
// Q12 Return all subjects in lowercase
// Output -> [flutter, dart, firebase]
List<String> lowerSubjects(List<String> arr) {}
```

```
// Q13 Return highest number ignoring null values
// Output -> 40
int highestNumber(List<int?> arr) {}
```



```

// Q14 Promise/Future
// Return "Passed" after 2 sec if total numbers > 50
// otherwise return "Failed"
// Output -> Passed
Future<String> resultStatus(List<int?> arr) {}

// Q15 Consume resultStatus using then()
// print result in lowercase
// Output -> passed
void resultReport(List<int?> arr) {}

// DRIVER FUNCTION DO NOT MODIFY
void driver() {
    print("Q1: ${subjectCount(subjects)}");
    print("Q2: ${firstSubject(subjects)}");
    print("Q3: ${studentCity(stu)}");
    print("Q4: ${squareNumbers(numbers)}");
    print("Q5: ${totalNumbers(numbers)}");
    print("Q6: ${evenGreaterThan15(numbers)}");
    print("Q7: ${calculator(20, 10, 'mul')}");
    print("Q8: ${studentInfo(stu)}");
    print("Q9: ${cgpaStatus(stu)}");
    print("Q10: ${scholarshipAmount()}");
    print("Q11: ${mixinDemo()}");
    print("Q12: ${lowerSubjects(subjects)}");
    print("Q13: ${highestNumber(numbers)}");
    resultStatus(numbers).then((value) => print("Q14: $value"));
    resultReport(numbers);
}

void main() {
    driver();
} // Copy Till Here

```

Q.3 Student Result and Scholarship System using Flask (20 Marks)

Upload app.py, style.css, index.html

Create a Flask web application using:

- Flask Routing
- Jinja Templates
- HTML Forms
- CSS Styling

Requirements

1. Home Page (/)

Create a form that accepts the following details:

- Roll Number
- Password
- Internal Marks
- External Marks

After clicking the submit button, data should be sent to /result route.

2. Use Hardcoded Dictionary

Use the following dictionary in app.py instead of database:

```
students = {  
    "101": {"password": "abc123", "name": "Rahul", "attendance": 85},  
    "102": {"password": "xyz789", "name": "Priya", "attendance": 60},  
    "103": {"password": "pass111", "name": "Aman", "attendance": 92}  
}
```

3. Login Verification

Check:

- Roll Number exists
- Password matches

If login fails, display:

Invalid Roll Number or Password in red color.

4. Calculation Logic

If login is successful:

Calculate:

$\text{total} = \text{internal} + \text{external}$

$\text{percentage} = \text{total} / 2$

Apply the following conditions:

if $\text{percentage} \geq 75$ and $\text{attendance} \geq 75$:

$\text{status} = \text{"Scholarship Eligible"}$

otherwise $\text{percentage} \geq 40$:

$\text{status} = \text{"Pass"}$

otherwise

$\text{status} = \text{"Fail"}$

5. Result Page

Display the following:

- Student Name
 - Roll Number
 - Attendance
 - Total Marks
 - Percentage
 - Final Status
-

6. CSS Requirements

Create a separate CSS file inside static folder.

Use CSS for:

- Background colour

- Centered card/container
 - Input styling
 - Button styling
 - Hover effect on button
 - Border radius
 - Box shadow
-

7. Conditional Styling using Jinja

Display status using different colours:

- Green → Scholarship Eligible
 - Orange → Pass
 - Red → Fail
-

8. Project Structure

```
flask_project/
├── app.py
├── static/
│   └── style.css
├── templates/
│   ├── index.html
│   └── result.html
```

Output Example

Successful Login:

- Displays student result with styled output page.

Failed Login:

- Displays red coloured error message.
-

Q.4 Question 4 contains two options:

(a) Django

20 Marks

OR

(b)

Part (i): ES6

10 Marks

Part (ii): Dart

10 Marks

Students have to attempt either option (a) or option (b).

Question 4(a) — Django Web Development

Question: Movie Ticket Booking and Bill Generation System using Django

[Upload style.css, booking.html and views.py]

Create a Django web application using:

- Django Templates
- Routing
- Forms
- CSS Styling
- Hardcoded Dictionary Data

Requirements

1. Booking Page (/)

Create a form that accepts:

- Customer Name

- Movie Name
- Number of Tickets
- Coupon Code

After clicking the submit button, data should be sent to /bill.

2. Use Hardcoded Dictionary

Use the following dictionary in views.py instead of database:

```
movies = {  
    "Avengers": {"price": 250, "available_seats": 50},  
    "Pushpa": {"price": 180, "available_seats": 30},  
    "KGF": {"price": 200, "available_seats": 0}  
}
```

3. Booking Validation

Check:

- Movie exists
- Seats are available
- Requested tickets should not exceed available seats

If booking fails display:

- “Movie Not Available”
OR
- “Seats Not Available”

in red color.

4. Bill Calculation

If booking is successful:

total = price * tickets

Apply discount:

```
if coupon == "SAVE20":  
    discount = total * 0.20  
else:  
    discount = 0
```

Final bill calculation:

```
final_amount = total - discount
gst = final_amount * 0.18
grand_total = final_amount + gst
```

5. Bill Page

Display the following:

- Customer Name
 - Movie Name
 - Number of Tickets
 - Ticket Price
 - Discount
 - GST
 - Final Amount
-

6. CSS Requirements

Create a separate CSS file inside static folder.

Use CSS for:

- Background color
 - Centered container/card
 - Input styling
 - Button styling
 - Hover effect on button
 - Border radius
 - Box shadow
 - Styled bill section
-

7. Conditional Styling using Django Templates

Display:

- Green → Booking Successful
- Red → Error Messages
- Blue → Final Bill Amount

8. Project Structure

```
django_project/
├── manage.py
├── static/
│   └── style.css
├── templates/
│   ├── booking.html
│   └── bill.html
└── movieapp/
    ├── views.py
    └── urls.py
```

Output Example

Successful Booking:

- Displays movie booking bill with styled output page.

Failed Booking:

- Displays red colored error message.

Question 4(b) — ES6 and Dart

(20 Marks)

(1) ES6 Question

(10 Marks)

//Copy Code From Here

```
const movies = [
  {
    title: "Inception",
    rating: 8.8,
    genres: ["Sci-Fi", "Action"],
    earnings: [120, 150, 200],
    details: {
      director: "Christopher Nolan",
```



```

        year: 2010
    },
    {
        title: "Avengers",
        rating: 8.0,
        genres: ["Action", "Adventure"],
        earnings: [300, 250, 400],
        details: null
    },
    {
        title: "Interstellar",
        rating: 8.7,
        genres: ["Sci-Fi", "Drama"],
        earnings: [180, 220, 260],
        details: {
            director: "Christopher Nolan",
            year: 2014
        }
    }
];

const platforms = ["Netflix", "Prime", "Hotstar", "JioCinema"];

```

```

// Q1 Return total number of movies
// Output -> 3
function totalMovies(arr){
}

```

```
// Q2 Return first platform in lowercase
```

```
// Output -> netflix
```

```
function firstPlatform(arr){
```

```
}
```

```
// Q3 Return title of first movie
```

```
// Output -> Inception
```

```
function firstMovie(arr){
```

```
}
```

```
// Q4 Return First movie rating status
```

```
// if rating > 8.5 return Blockbuster otherwise Hit
```

```
// Output -> Blockbuster
```

```
function movieStatus(arr){
```

```
}
```

```
// Q5 Return movie titles sorted by rating descending
```

```
// Output -> ['Inception','Interstellar','Avengers']
```

```
function sortByRating(arr){
```

```
}
```

```
// Q6 Return all genres in uppercase
// Output -> ['SCI-FI','ACTION','ACTION','ADVENTURE','SCI-FI','DRAMA']
function upperGenres(arr){

}

// Q7 Return movies whose average earnings are above 200
// Output -> ['Avengers','Interstellar']
function profitableMovies(arr){
}

// Q8 Return director names
// if details not available return "Director Not Found"
// Output -> ['Christopher Nolan','Director Not Found','Christopher Nolan']
function movieDirectors(arr){
}

// Q9 Promise resolve after 2 sec
// if average movie rating > 8 return "Platform Successful"
// otherwise return "Platform Average"
// Output -> Platform Successful
function platformStatus(arr){
}

// Q10 Consume platformStatus using then()
// print result in lowercase
// Output -> platform successful
function platformReport(arr){

}
```

```
// DRIVER FUNCTION DO NOT MODIFY
```

```
function driver(){

    console.log("Q1:", totalMovies(movies));
    console.log("Q2:", firstPlatform(platforms));
    console.log("Q3:", firstMovie(movies));
    console.log("Q4:", movieStatus(movies));

    console.log("Q5:", sortByRating(movies));
    console.log("Q6:", upperGenres(movies));
    console.log("Q7:", profitableMovies(movies));
    console.log("Q8:", movieDirectors(movies));

    platformStatus(movies)
        .then(res => console.log("Q9:", res))
        .catch(err => console.log("Q9:", err));

    platformReport(movies);

}

driver();
```

(2) Dart Question

(10 Marks)

```
//Copy From Here
```

```
List<int?> bills = [5000, null, -1200, 3000, 7000, null, 2500];
```

```
List<String> departments = ["Cardiology", "Neurology", "Orthopedic"];
```

```
class Patient {  
    static String hospitalName = "City Hospital";  
  
    String name;  
    int patientId;  
  
    double _healthScore;  
  
    String? city;  
  
    Patient(this.name, this.patientId, this._healthScore, this.city);  
  
    // Getter  
    double get healthScore => _healthScore;  
  
    // Setter  
    set healthScore(double value) {  
        if (value >= 0 && value <= 10) {  
            _healthScore = value;  
        }  
    }  
}  
  
class VIPPatient extends Patient {  
    double insuranceAmount;  
  
    VIPPatient(  
        String name,  
        int patientId,  
        double healthScore,  
        String? city,  
        this.insuranceAmount,
```

```
    ) : super(name, patientId, healthScore, city);  
}
```

```
abstract class MedicalReport {  
    String reportStatus();  
}
```

```
class DoctorReport extends MedicalReport {  
    @Override  
    String reportStatus() {  
        return "Report Generated";  
    }  
}
```

```
mixin Emergency {  
    String emergencyService() {  
        return "Emergency Service Active";  
    }  
}
```

```
mixin Ambulance {  
    String ambulanceService() {  
        return "Ambulance Ready";  
    }  
}
```

```
class Hospital with Emergency, Ambulance {}
```

```
Patient p1 = Patient("Harsh", 201, 8.5, null);
```

// Q1 Return hospital name using static variable

// Output -> City Hospital

String hospitalNameDemo() {}

// Q2 Return patient city

// if city is null return "City Not Available"

// Output -> City Not Available

String patientCity(Patient p) {}

// Q3 Use getter method to return healthScore

// Output -> 8.5

double getterDemo(Patient p) {}

// Q4 Use setter method to update healthScore to 9.2

// Output -> 9.2

double setterDemo(Patient p) {}

// Q5 Return all valid bills

// ignore null values and convert negative into positive

// Output -> [5000, 1200, 3000, 7000, 2500]

List<int> validBills(List<int?> arr) {}

// Q6 Return total bill amount (convert negative to positive)

// Output -> 18700

int totalBills(List<int?> arr) {}

// Q7 Create VIPPatient object

// return insurance amount

// Output -> 100000

double insuranceDetails() {}

```

// Q8 Use abstract class implementation
// Output -> Report Generated
String reportDemo() {}

// Q9 Use multiple mixins
// Output -> Emergency Service Active & Ambulance Ready
String hospitalServices() {}

// Q10 Future
// after 2 sec return "Hospital Running Successfully"
// if total bill > 15000 otherwise "Low Revenue"
// print result in lowercase using then()
// Output -> q10: hospital running successfully
void hospitalReport(List<int?> arr) {}

// DRIVER FUNCTION DO NOT MODIFY
void driver() {
    print("Q1: ${hospitalNameDemo()}");
    print("Q2: ${patientCity(p1)}");
    print("Q3: ${getterDemo(p1)}");
    print("Q4: ${setterDemo(p1)}");
    print("Q5: ${validBills(bills)}");
    print("Q6: ${totalBills(bills)}");
    print("Q7: ${insuranceDetails()}");
    print("Q8: ${reportDemo()}");
    print("Q9: ${hospitalServices()}");
    hospitalReport(bills);
}

void main() {
    driver();
} // Copy Till Here

```